RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE

École Supérieure en Sciences et Technologies
de l’Informatique et du Numérique

ANALYSE ET CONCEPTION
DU PROJET

Agent métier — Marketing Automation

Équipe	VisionX
Module	Mini-projet « Conduite de projet »
Niveau	2ème année CS
Document	Analyse fonctionnelle, architecture et feuille de route

Version de travail — 07 novembre 2026

Sommaire

1. 1. Présentation générale du projet
2. 2. Analyse des exigences
3. 3. Périmètre fonctionnel proposé
4. 4. Architecture technique
5. 5. Technologies et outils
6. 6. Modèle de données préliminaire
7. 7. API prévisionnelle
8. 8. Interface utilisateur proposée
9. 9. Sécurité et qualité
10. 10. Organisation de l’équipe VisionX
11. 11. Planification et livrables L1 à L8
12. 12. Structure du dépôt proposée
13. 13. Risques, inconnues et décisions à prendre
14. 14. Conclusion et prochaines étapes

1. 1. Présentation générale du projet

Le mini-projet porte sur la conception et le développement d'agents logiciels intégrés à une plateforme commune. Chaque équipe doit développer un seul agent, qui doit fonctionner de manière autonome et respecter les contrats d'intégration fournis par l'enseignant.

L'équipe VisionX a choisi la catégorie « agent d'automatisation métier », et plus précisément l'agent « Marketing automation ». Selon le sujet, sa fonction principale concerne les campagnes emailing et les réseaux sociaux automatisées.

Objectif central : développer un agent Marketing Automation autonome, capable de s'intégrer au noyau commun, de produire les objets métier « prospect » et de consommer les objets métier « commande », conformément au registre et aux contrats officiels.

1.1 Comprendre simplement le projet

Le marketing automation consiste à automatiser certaines opérations marketing habituellement réalisées manuellement : gérer des prospects, préparer des campagnes, planifier des communications et déclencher des actions selon des événements ou des règles.

Exemple illustratif : une entreprise enregistre un prospect. Une campagne peut ensuite lui adresser un message de bienvenue. Si une commande est publiée par la plateforme, l'agent reçoit l'événement et peut adapter le parcours marketing. Les règles exactes de déclenchement devront être définies par l'équipe et rester conformes aux contrats.

1.2 Objectifs

- Développer un agent métier spécialisé dans l'automatisation marketing.
- Permettre la gestion des prospects et des campagnes.
- Mettre en place des mécanismes d'automatisation des communications.
- Respecter les contrats techniques et le registre des objets métier.
- Assurer une intégration fonctionnelle avec le noyau central.
- Fournir une image Docker autonome et une démonstration réelle.

1.3 Acteurs et responsabilités

Acteur	Responsabilité
Utilisateur de la plateforme	Utilise les fonctions marketing autorisées par son compte.
Équipe VisionX	Conçoit, développe, teste et documente l'agent.
Noyau central	Fournit l'authentification, la gestion des utilisateurs/entreprises, le catalogue d'agents et la passerelle d'intégration.
Enseignant / assistant	Fournit les contrats, le dépôt privé et évalue les livrables.

2. 2. Analyse des exigences

2.1 Exigences confirmées par les documents

Élément	Exigence issue du sujet
Catégorie	Agent d'automatisation métier.

Élément	Exigence issue du sujet
Agent choisi	Marketing automation.
Fonction indiquée	Campagnes emailing et réseaux sociaux automatisées.
Technologies proposées	Mautic, Postfix (SMTP), Python.
Objet produit	prospect.
Objet consommé	commande.
Contrat	Les types d'objets produits/consommés doivent correspondre au registre INTER_AGENT_DATA_CONTRACT.
Intégration	La collaboration entre agents passe exclusivement par le noyau.
Code source	Chaque équipe doit conserver le cloisonnement du code source.
Dépôt	Utiliser exclusivement le dépôt privé fourni par l'enseignant.
Livraison	Fournir au minimum le code complet commenté, agent.manifest.json, Dockerfile et README à la date T6.

2.2 Exigences techniques obligatoires visibles dans le sujet

- Fournir le fichier agent.manifest.json conforme au schéma imposé, avec les champs produces/consumes pour les agents métier.
- Implémenter les endpoints obligatoires : /health, /tenant/activate, /tenant/deactivate, /tenant/{id}/status et /config-schema.
- Respecter l'enveloppe de réponse standard {status, data, error}.
- Publier/recevoir les objets métier selon le registre via les mécanismes /exchange/... prévus par le noyau.
- Utiliser /exchange/notify si l'agent consomme des objets métier, selon les indications du sujet.
- Assurer l'isolation stricte des données par utilisateur/entreprise.
- Fournir un packaging sous forme d'image Docker fonctionnelle et autonome.

2.3 Exigences de collaboration

Un utilisateur peut souscrire à plusieurs agents. Lorsqu'un agent publie un objet métier, le noyau notifie les agents actifs du même utilisateur qui déclarent consommer ce type d'objet. Si l'utilisateur n'a pas souscrit à l'agent producteur, il peut alimenter ou consulter le flux depuis le tableau de bord, selon les règles de la plateforme.

Les agents ne doivent jamais connecter directement leurs codes entre eux. Le noyau est l'intermédiaire obligatoire.

2.4 Exigences proposées, à valider

- Gestion CRUD des prospects, avec recherche, filtres et segmentation.
- Création, planification, activation et suspension des campagnes.
- Gestion de modèles d'emails.
- Historique des envois et suivi des statuts.
- Tableau de bord avec indicateurs marketing.
- Import CSV des prospects, si cela est autorisé et utile.
- Gestion du consentement et des désinscriptions.
- Traitement des événements de commande pour adapter les communications.

Ces fonctionnalités constituent une proposition de périmètre fonctionnel. Elles ne sont pas toutes explicitement imposées par les pages photographiées.

3. 3. Périmètre fonctionnel proposé

3.1 Module de gestion des prospects

But : gérer les personnes ou contacts susceptibles de devenir clients et produire les objets métier de type prospect selon le format officiel.

- Créer, consulter, modifier et archiver un prospect.
- Enregistrer les coordonnées, la source et le statut du prospect, uniquement selon les champs autorisés.
- Rechercher, filtrer et segmenter les prospects.
- Valider les champs obligatoires et éviter les doublons.
- Publier les objets prospect via le noyau, jamais par une connexion directe à un autre agent.

3.2 Module de gestion des campagnes

- Créer une campagne et définir son nom, son objectif et son contenu.
- Choisir une audience ou un segment de prospects.
- Préparer et prévisualiser le message.
- Planifier, lancer, suspendre ou terminer une campagne selon les droits.
- Consulter l'état et l'historique de la campagne.

3.3 Module d'automatisation emailing

- Configurer des déclencheurs et des actions marketing.
- Utiliser Mautic comme moteur de marketing automation si son intégration est confirmée.
- Utiliser Postfix/SMTP pour le transport des emails selon la configuration retenue.
- Journaliser les tentatives d'envoi et les erreurs.
- Prévoir les reprises contrôlées en cas d'échec et éviter les envois en double.

3.4 Module d'intégration des commandes

L'agent doit déclarer qu'il consomme l'objet métier commande. Il reçoit les événements par le mécanisme du noyau, valide leur structure selon le contrat, vérifie le tenant concerné et traite l'événement de manière idempotente.

Exemple proposé : après réception d'une commande, l'agent peut mettre à jour le contexte marketing d'un contact ou exclure ce contact d'une campagne de prospection. Le comportement précis dépendra des champs réellement disponibles dans l'objet commande.

3.5 Module de reporting

- Afficher le nombre de prospects et leur évolution.
- Présenter le nombre de campagnes par état.
- Suivre les envois, échecs et autres métriques effectivement fournies par Mautic ou le serveur email.
- Ne pas présenter les ouvertures ou clics comme fiables si le mécanisme de collecte n'est pas configuré.

3.6 Priorisation

Fonctionnalité	Priorité proposée	Statut
Conformité au contrat et au noyau	Critique	Obligatoire
Manifest et objets métier	Critique	Obligatoire
Gestion des prospects	Haute	Cœur métier proposé
Réception des commandes	Haute	Obligatoire au registre
Campagnes marketing	Haute	Cœur métier proposé
Email automation	Haute	Fonction principale
Reporting	Moyenne	Extension proposée
Réseaux sociaux	Moyenne	À préciser selon le périmètre

4. 4. Architecture technique

4.1 Principe d’architecture

Architecture recommandée : application web client-serveur, backend Python indépendant, organisation modulaire en couches et API REST pour les échanges internes. Le noyau fourni par l’enseignant reste une dépendance externe : l’équipe VisionX ne doit pas le redévelopper.

Le choix d’un monolithe modulaire est adapté à une équipe étudiante et à un agent unique : il permet de séparer clairement les domaines fonctionnels sans introduire la complexité de plusieurs microservices.

4.2 Architecture logique

```
Utilisateur
    |
    v
Noyau commun (authentification, tenants, catalogue, passerelle)
    |
    | notifications / échanges métier via contrats officiels
    v
Agent VisionX Marketing Automation
    |
    +--> API Backend Python / Django REST Framework
    |       +--> Module Prospects
    |       +--> Module Campagnes
    |       +--> Module Automatisation
    |       +--> Module Reporting
    |       +--> Adaptateur du noyau
    |
    +--> PostgreSQL (données propres à l’agent)
    |
    +--> Mautic (campagnes / scénarios, si retenu)
             |
             v
         Postfix / SMTP
             |
             v
       Destinataires email

Frontend React + Vite <---- API REST ----> Backend
```

4.3 Architecture frontend

- React + Vite pour l’application web.
- Pages organisées par domaine : dashboard, prospects, campagnes, automatisations, analytics et paramètres.

- Composants réutilisables : tableaux, formulaires, filtres, modales, cartes KPI et notifications.
- Un module de services centralise les appels HTTP vers le backend.
- Les états de chargement, erreurs, accès refusés et listes vides doivent être gérés.

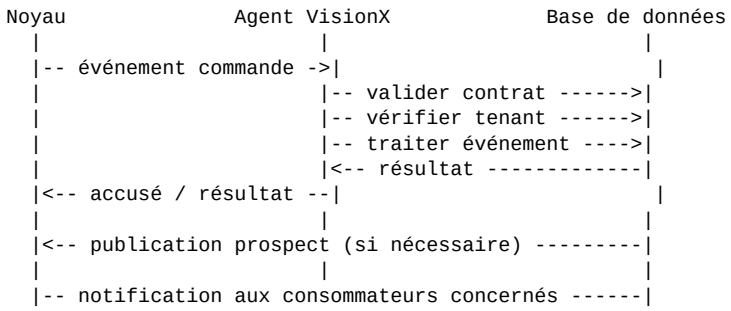
4.4 Architecture backend

- Django REST Framework expose les API métier.
- Les vues/controllers reçoivent les requêtes HTTP et délèguent la logique aux services.
- Les serializers valident les données entrantes et formatent les réponses.
- Les modèles décrivent les données locales de l'agent.
- Un adaptateur dédié communique avec le noyau en respectant le contrat.
- Les tâches longues (campagnes, envois) peuvent être exécutées en arrière-plan si le besoin est confirmé.

4.5 Architecture infrastructure

- Docker pour empaqueter l'agent et ses dépendances.
- Docker Compose pour l'environnement de développement local, si autorisé.
- PostgreSQL pour les données métier locales.
- Variables d'environnement pour les secrets et paramètres.
- Journaux applicatifs pour diagnostiquer les erreurs.
- Sauvegardes de la base et procédure de restauration à définir.

4.6 Diagramme de séquence illustratif



Le diagramme illustre le principe d'échange. Les URL exactes, méthodes HTTP, authentification et formats doivent être extraits du contrat officiel.

5. 5. Technologies et outils

Élément	Choix recommandé / proposé	Rôle
Frontend	React + Vite	Interface web.
Langage frontend	JavaScript ou TypeScript	Logique de l'interface.
Backend	Python + Django REST Framework	API et logique métier.
Base de données	PostgreSQL	Stockage relationnel.
Marketing automation	Mautic	Gestion de campagnes et scénarios, si retenu.
Email	Postfix / SMTP	Transport des emails.

Élément	Choix recommandé / proposé	Rôle
Conteneurisation	Docker / Docker Compose	Packaging et environnement local.
Versionnement	Git + dépôt privé GitHub fourni	Historique et collaboration.
Tests API	Postman	Vérification des endpoints.
IDE	VS Code ou IDE équivalent	Développement.

5.1 Outils de travail

Outil	Utilité	Statut
VS Code	Édition du code et extensions.	Recommandé
Git	Historique des modifications.	Obligatoire en pratique
GitHub privé fourni	Dépôt officiel de l'équipe.	Obligatoire
Postman	Tests des API.	Recommandé
Docker	Construction et exécution du conteneur.	Obligatoire pour le packaging
DBeaver ou pgAdmin	Inspection de PostgreSQL.	Optionnel
Figma	Maquettes et collaboration UI.	Optionnel
Trello / GitHub Projects	Suivi des tâches.	Optionnel
LaTeX / Word	Rédaction des livrables.	Selon consignes

5.2 Pourquoi éviter le surdimensionnement

Il est préférable de commencer par un seul backend modulaire et une base PostgreSQL. Il ne faut pas ajouter Kubernetes, une architecture microservices ou plusieurs brokers de messages sans besoin explicite. Le noyau fournit déjà une passerelle d'intégration : votre agent doit utiliser cette interface plutôt que créer un mécanisme parallèle.

6. 6. Modèle de données préliminaire

Le modèle ci-dessous est une proposition interne. Il ne remplace pas le schéma officiel des objets métier du document INTER_AGENT_DATA_CONTRACT. Les champs exacts de prospect et commande restent inconnus tant que le contrat n'a pas été consulté.

Entité	Champs indicatifs	Rôle
Prospect	id, tenant_id, email, nom, source, statut, created_at	Contact suivi par l'agent.
Campaign	id, tenant_id, nom, objectif, statut, scheduled_at	Campagne marketing.
CampaignContact	id, campaign_id, prospect_id, statut	Association prospect-campagne.
EmailLog	id, campaign_id, prospect_id, état, sent_at, error	Historique des tentatives d'envoi.
AutomationRule	id, campaign_id, trigger_type, action_type, enabled	Règle d'automatisation.

6.1 Relations

```
PROSPECT 1 ----- N CAMPAIGN_CONTACT N ----- 1 CAMPAIGN
   |                                          |
   |                                          +---- N EMAIL_LOG
   +------------------------- 1 ----- N EMAIL_LOG  |
                                              |
CAMPAIGN 1 ----- N AUTOMATION_RULE
```

6.2 Contraintes de conception

- Chaque donnée doit être rattachée au tenant approprié.
- Les identifiants et contraintes uniques doivent être choisis selon les règles du métier.
- Les suppressions doivent tenir compte de l'historique des campagnes et des obligations de conservation.
- Les événements reçus doivent être enregistrés avec un identifiant permettant de prévenir les traitements en double.
- Les champs officiels des objets partagés ne doivent pas être inventés ni modifiés unilatéralement.

7. 7. API prévisionnelle

Les endpoints ci-dessous sont des routes internes proposées pour l'API métier de VisionX. Ils ne sont pas les endpoints du noyau et ne doivent pas être confondus avec les routes contractuelles obligatoires.

Méthode	Route interne proposée	Description
GET	/api/prospects/	Lister les prospects.
POST	/api/prospects/	Créer un prospect.
GET	/api/prospects/{id}/	Consulter un prospect.
PATCH	/api/prospects/{id}/	Modifier un prospect.
DELETE	/api/prospects/{id}/	Archiver/supprimer selon la règle retenue.
GET	/api/campaigns/	Lister les campagnes.
POST	/api/campaigns/	Créer une campagne.
GET	/api/campaigns/{id}/	Consulter une campagne.
PATCH	/api/campaigns/{id}/	Modifier une campagne.
POST	/api/campaigns/{id}/launch/	Lancer une campagne.
POST	/api/campaigns/{id}/pause/	Suspendre une campagne.
GET	/api/analytics/overview/	Consulter les indicateurs globaux.

7.1 Routes contractuelles du sujet

Endpoint cité dans le sujet	Fonction
/health	Vérifier l'état de fonctionnement.
/tenant/activate	Activer un tenant.
/tenant/deactivate	Désactiver un tenant.
/tenant/{id}/status	Consulter l'état d'un tenant.

Endpoint cité dans le sujet	Fonction
/config-schema	Exposer le schéma de configuration.

Le sujet demande une enveloppe de réponse standard de forme {status, data, error}. La structure exacte des champs et les codes HTTP doivent être vérifiés dans AGENT_INTEGRATION_CONTRACT.doc.

7.2 Exemple indicatif d’enveloppe

```
{
  "status": "success",
  "data": {},
  "error": null
}
```

Exemple pédagogique uniquement : ne pas considérer cet exemple comme la spécification complète du contrat.

8. 8. Interface utilisateur proposée

Les photographies du sujet ne donnent pas de maquette UI imposée. Les pages suivantes sont donc une proposition pour rendre l’agent utilisable et démontrable.

Page	Contenu et actions
Dashboard	Vue synthétique, KPI, campagnes récentes et activité.
Prospects	Tableau, recherche, filtres, création et modification.
Détail prospect	Informations du contact, statut et historique.
Campagnes	Liste, statut, recherche et actions.
Créer une campagne	Nom, objectif, audience, contenu et planification.
Modèles email	Créer et gérer les modèles de messages.
Automatisations	Règles, déclencheurs, actions et activation.
Analytics	Résultats et indicateurs disponibles.
Historique email	Tentatives, états, dates et erreurs.
Paramètres	Configuration de l’agent et intégrations autorisées.

8.1 Principes UX

- Navigation simple et cohérente.
- Formulaires avec validation claire.
- Filtres et pagination pour les listes.
- Confirmation avant les actions sensibles.
- États visuels pour brouillon, actif, suspendu, terminé et erreur.
- Interface responsive et messages d’erreur compréhensibles.

9. 9. Sécurité et qualité

Risque	Mesure recommandée
Accès non autorisé	Authentification et permissions selon le mécanisme du noyau et le contrat.
Fuite entre entreprises	Isolation stricte par tenant et tests croisés.
Injection SQL	ORM/paramétrage des requêtes et validation des entrées.
XSS / CSRF	Échappement des sorties, protections adaptées au mode d'authentification.
Secrets SMTP exposés	Variables d'environnement et stockage sécurisé.
Abus d'API	Limitation de débit si nécessaire et contrôle des permissions.
Email envoyé en double	Idempotence, identifiant d'événement et suivi des traitements.
Fichiers importés dangereux	Validation du type, taille, contenu et traitement sécurisé.
Perte de données	Sauvegardes et procédure de restauration.

9.1 Tests à prévoir

- Tests unitaires des services prospects, campagnes et règles.
- Tests des serializers et validations.
- Tests d'intégration avec PostgreSQL.
- Tests API des routes internes.
- Tests contractuels des endpoints et réponses du noyau.
- Tests d'isolation entre tenants.
- Tests de réception d'une commande et de publication d'un prospect.
- Tests d'échec SMTP, de reprise et de prévention des doublons.
- Tests de bout en bout du scénario de démonstration.

10. 10. Organisation de l'équipe VisionX

Le sujet prévoit des équipes de cinq à six personnes. Les noms des membres ne sont pas reproduits ici : les responsabilités ci-dessous peuvent être affectées aux membres de l'équipe.

Rôle	Responsabilités
Chef de projet / architecte	Coordination, architecture, suivi des contrats, arbitrages et intégration globale.
Backend 1	Module prospects, modèles, API et validations.
Backend 2	Campagnes, automatisations et logique métier.
Frontend	Dashboard React, pages, composants et intégration API.
Intégration / DevOps	Mautic, SMTP, Docker, configuration et communication avec le noyau.
QA / documentation	Plan de tests, vérification, rapports et préparation de la démonstration.

Si l'équipe compte cinq personnes, certaines responsabilités peuvent être cumulées. Chaque tâche doit avoir un responsable principal et un livrable vérifiable.

10.1 Workflow Git recommandé

```
main
develop
feature/prospects
feature/campaigns
feature/email-automation
feature/frontend
feature/integration
feature/docker
```

- Utiliser exclusivement le dépôt privé fourni par l'enseignant.
- Ajouter l'enseignant et l'assistant comme collaborateurs dès la création du dépôt.
- Ne jamais rendre le dépôt public et ne jamais inviter un membre d'une autre équipe.
- Ne pas copier le code source d'une autre équipe.
- Faire des commits réguliers et explicites.
- Fusionner les fonctionnalités après revue et tests.

11. 11. Planification et livrables L1 à L8

Livrable	Contenu demandé	Volume / échéance
L1	Document d'orientation : agent choisi, objectif, justification, méthodologie, plan, sources, contrat d'intégration, objets partagés, risques.	Environ 1 page, Word, avant T2.
L2	Charte : définition, objectifs, bénéfices, livrables, tâches, ressources, affectations et analyse des risques.	6–7 pages, Arial 10/12, interligne 1,5, avant T3.
L3	PV de lancement : rôles et engagement collectif sur le contrat.	1–2 pages, avant T4.
L4	Planification WBS/OBS/RACI, échéancier, jalons, dépendances, PERT, Gantt, coûts et efforts.	5–6 pages, avant T5.
L5	Rapport d'exécution : langages, frameworks, outils, versions, problèmes, conformité et dépôt GitHub.	6–7 pages hors annexes, avant T6.
L6	Rapport de suivi : KPI prévisionnels/réels et état de conformité.	3–5 pages hors annexes, avant T7.
L7	Bilan : réalisations, écarts, perspectives et leçons apprises.	4–5 pages hors annexes, avant T8.
L8	Présentation PowerPoint et démonstration réelle de l'agent actif sur le noyau.	20 min + 5 min questions, avant T9.

11.1 Proposition de phases de développement

Phase	Travaux	Critère de fin
1. Analyse	Lire les contrats, clarifier le périmètre et rédiger L1.	Exigences et contrats compris.
2. Conception	Architecture, données, API, UX, L2 et L3.	Conception validée par l'équipe.
3. Planification	WBS, OBS, RACI, Gantt, PERT et L4.	Tâches et responsables définis.
4. Socle technique	Dépôt, environnement, Django, React, PostgreSQL et Docker.	Projet démarre localement.

Phase	Travaux	Critère de fin
5. Fonctionnalités métier	Prospects, campagnes et automatisations.	Scénarios métier testés.
6. Intégration	Manifest, endpoints, objets partagés et noyau.	Tests contractuels réussis.
7. Qualité	Sécurité, tests, corrections et suivi L5/L6.	Agent stable et conforme.
8. Livraison	Bilan, documentation, démonstration et L7/L8.	Démonstration fonctionnelle.

Les dates calendaires et durées précises ne peuvent pas être établies à partir des pages reçues : elles doivent être renseignées selon le calendrier réel des séances T1 à T9.

12. 12. Structure du dépôt proposée

```
visionx-marketing-agent/
■■■ backend/
■   ■■■ config/
■   ■■■ apps/
■   ■   ■■■ prospects/
■   ■   ■■■ campaigns/
■   ■   ■■■ email_automation/
■   ■   ■■■ analytics/
■   ■   ■■■ integrations/
■   ■■■ core/
■   ■■■ manage.py
■   ■■■ requirements.txt
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■■■ pages/
■   ■   ■■■ layouts/
■   ■   ■■■ services/
■   ■   ■■■ hooks/
■   ■   ■■■ routes/
■   ■■■ package.json
■■■ infrastructure/
■   ■■■ docker-compose.yml
■■■ database/
■■■ tests/
■■■ docs/
■■■ agent.manifest.json
■■■ Dockerfile
■■■ README.md
```

Cette arborescence est indicative. Elle pourra être ajustée après lecture du guide développeur et du schéma de manifest.

13. 13. Risques, inconnues et décisions à prendre

Point	État	Action nécessaire
Format exact de prospect	Inconnu	Lire INTER_AGENT_DATA_CONTRACT.doc.
Format exact de commande	Inconnu	Lire INTER_AGENT_DATA_CONTRACT.doc.
Schéma complet du manifest	Inconnu	Lire AGENT_INTEGRATION_CONTRACT.doc et le guide.

Point	État	Action nécessaire
Authentification entre noyau et agent	Inconnu	Vérifier les exigences du contrat.
Méthodes HTTP et corps des endpoints tenant	Partiellement connus	Vérifier les signatures officielles.
Configuration de Mautic	À décider	Évaluer l'intégration et l'environnement disponible.
Réseaux sociaux	À préciser	Définir les plateformes et API accessibles.
Mesures analytics disponibles	À préciser	Confirmer les événements collectés.
Calendrier T1–T9	Non fourni	Récupérer les dates de séances et jalons.

13.1 Risques majeurs

- Non-conformité au contrat : peut empêcher le branchement de l'agent au noyau et affecter fortement L5 et L8.
- Retard d'intégration : réduit le temps disponible pour tester la démonstration finale.
- Configuration SMTP/Mautic : peut bloquer les scénarios d'envoi.
- Isolation insuffisante : risque de fuite de données entre entreprises.
- Coordination d'équipe : conflits de code et responsabilités non définies.

13.2 Documents à obtenir avant de figer la conception

- AGENT_INTEGRATION_CONTRACT.doc : contrat technique obligatoire (endpoints, formats, sécurité).
- INTER_AGENT_DATA_CONTRACT.doc : registre des objets métier partagés, notamment prospect et commande.
- DEVELOPER_INSTRUCTIONS.doc : guide pas-à-pas pour développer et soumettre l'agent.

14. 14. Conclusion et prochaines étapes

Le projet VisionX consiste à développer un agent Marketing Automation autonome, spécialisé dans les campagnes emailing et les réseaux sociaux automatisées, et intégré à une plateforme commune par l'intermédiaire du noyau central.

Le sujet confirme que l'agent doit produire prospect et consommer commande. Il impose également le respect des endpoints de cycle de vie, du manifest, des échanges via le noyau, de l'isolation des données, du packaging Docker et du dépôt privé.

Priorité absolue : obtenir et analyser les trois documents contractuels avant de figer les modèles de données, le manifest, les routes d'intégration et le code.

Actions immédiates pour VisionX

- Récupérer les trois documents contractuels auprès de l'enseignant.
- Confirmer la composition de l'équipe et affecter les responsabilités.
- Rédiger L1 en utilisant les exigences confirmées et en signalant les points inconnus.
- Créer le dépôt privé fourni et organiser le travail Git.
- Valider l'architecture avec les contrats avant de commencer le développement complet.
- Construire d'abord un prototype minimal qui se branche au noyau, puis ajouter les fonctionnalités marketing.